

Written exam, Functional Programming

THURSDAY MAY 16, 2019

- The exam duration is four hours
- There are four questions. To obtain full marks you must answer all the subquestions satisfactorily
- You are allowed to use books, lecture notes, lecture slides, hand-ins, solutions to assignments, calculators, computers, software, on-line resources etc. during the examination. This includes any form of device that can execute programs written in F#.
- You may **NOT** copy code found online that you yourself have not written and hand that in as your solution. To be safe stick to resources such as *F# for fun and profit* or Microsoft's documentation of .NET and the F# language.
- You are (unless otherwise instructed) allowed to use the .NET library including the modules described in the book, e.g., List, Set, Map etc.
- If a subquestion requires you to define a particular function, then you may (unless otherwise instructed) use that function in subsequent subquestions, even if you have not managed to define it. Providing the signature of the missing function will help in such cases.
- If a subquestion requires you to define a particular function, then you may (unless otherwise instructed) define as many helper functions as you want, but in any case you must define the required function so that it has exactly the type and effect that the subquestion asked for.
- The grading for the first three questions require functional solutions, and solutions with side effects will not be considered. The fourth question may use side effects for partial credit, but you are strongly encouraged to use functional solutions throughout the exam. The one exception to this rule concerns parallelism as `Async.Parallel` returns the results of the individual processes in an array and these results may be used.
- You should hand in one ASCII .fs or .fsx file only, e.g. `SWU_FP2019.fs`. If you hand in a .fs-file then it should use the namespace `SWU_FP2019`. Other file formats than these two will not be accepted.
- You are strongly encouraged, but not required, to use the provided code skeleton `SWU_FP2019.fs` as a basis for your submission. Any helper functions that are provided in this file may also be part of your submission.
- Most functions that you need to write are present in the code skeleton. If an assignment asks that you write a function `isEven : int -> bool`, for instance, then there is nearly always a corresponding `let isEven _ = failwith "Not implemented"` in the source file. You may change these functions (changing a `let` to a `let rec` for instance) as long as their signatures correspond to those given in the assignment. In this case that could be `let isEven x = x % 2 = 0`. Be wary of polymorphic variables as notation sometimes differs and some IDEs, for instance, will write `MyType<'a when 'a : equality>` while others may write `MyType<'a> when 'a : equality`. These are identical.

You MUST include explanations and comments to support your solutions for the questions that require them. You simply write them as comments around your code.

Your exam hand-in MUST be made by yourself and yourself only, and this holds for program code, examples, the explanation you provide for the code, and all other parts of the answers. It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes using solutions or code found online.

Your solution MUST compile. We reserve the right to fail any submission that does not meet this requirement.

1: Recursion on natural numbers (25%)

One way of encoding the natural numbers (the non-negative integers) is by using Peano numbers. These represent numbers in unary notation where `0` is zero and `S x` is the successor of `x` (i.e. `x + 1`). For example, `S (S (S 0))` is `3` and `S (S (S (S (S (S 0)))))` is `6`.

If this definition looks puzzling, then think of them as lists that store no values and that the number is the length of this list.

Peano numbers can be defined in the following manner:

```
type Peano =  
  | 0  
  | S of Peano
```

Question 1.1

Create the following two functions

1. `toInt : Peano -> uint32` that given a Peano number `a` returns the corresponding unsigned integer of `a`
2. `fromInt : uint32 -> Peano` that given an unsigned integer `x` returns the corresponding Peano number of `x`.

Important: These two functions are helpful for testing the rest of this assignment but they may not be used by any implementation in the exam.

Question 1.2

Create the following recursive (but not tail-recursive) functions.

1. `add : Peano -> Peano -> Peano` that given two Peano numbers `a` and `b` returns the sum of `a` and `b` as a Peano number.
2. `mult : Peano -> Peano -> Peano` that given two Peano numbers `a` and `b` returns the product of `a` and `b` as a Peano number.
3. `pow : Peano -> Peano -> Peano` that given two Peano numbers `a` and `b` returns `a` to the power of `b` as a Peano number.

Example:

```
add  (S (S O)) (S (S (S O))) = S (S (S (S (S O))))
mult (S (S O)) (S (S (S O))) = S (S (S (S (S (S O)))))
pow  (S (S O)) (S (S (S O))) = S (S (S (S (S (S (S (S O)))))))
```

In addition, observe the following:

- No library functions may be used
- The implementation of `add` may not use any auxiliary functions.
- The implementation of `mult` may only use `add` as an auxiliary function.
- The implementation of `pow` may only use `mult` as an auxiliary function.

Hint: $a * b = b + \dots a \text{ times } \dots + b$
 $a^b = b * \dots a \text{ times } \dots * b$

Question 1.3

Create tail-recursive versions of the functions from Question 1.2 (`tailAdd` , `tailMult` , and `tailPow`) that use accumulators (not continuations). The signatures and results must be the same and the same restrictions apply regarding library functions and auxiliary functions with the exception that you may use auxiliary functions to handle the accumulators.

For instance, `tailMult` may have an auxiliary function that handles the accumulator but other than that it may only use `tailAdd` as an auxiliary function.

Question 1.4

Create a function `loop : ('a -> 'a) -> 'a -> Peano -> 'a` that given a function `f` , a starting value `acc` , and a peano number `p` returns `f (... p times ... (f acc) ...)` (so the base case is `loop f acc 0 = acc` for any function `f`).

Example

The following examples returns `true` if a number is even and `false` if a number is odd.

```
loop not true 0 = true
loop not true (S (S (S O))) = false
loop not true (S (S (S (S (S (S (S (S O))))))) = true
```

Question 1.5

Create non-recursive functions (`loopAdd` , `loopMult` , and `loopPow`) that all use the `loop` function to create variants of the functions from 1.2. The signatures and results must be the same and the same restrictions apply regarding library functions and auxiliary functions with the exception that all of the functions must use `loop` .

For instance, `loopMult` must not be recursive, and may only use `loop` (which is recursive) and `loopAdd` as auxiliary functions.

2: Code comprehension (25%)

Consider the following two functions

```
let rec f x =  
  function  
  | []          -> None  
  | y::ys when x = y  -> Some ys  
  | y::ys when x <> y ->  
    match f x ys with  
    | Some ys' -> Some (y::ys')  
    | None     -> None  
  
let rec g xs =  
  function  
  | []      -> xs = []  
  | y::ys ->  
    match f y xs with  
    | Some xs' -> g xs' ys  
    | None     -> false
```

Question 2.1

- What are the types of functions `f` and `g` ?
- What do functions `f` and `g` do? Focus on what they do rather than how they do it.
- What would be appropriate names for functions `f` and `g` ?

Question 2.2

The function `f` generates a warning during compilation:

```
warning FS0025: Incomplete pattern matches on this expression.
```

- Why does this happen, and where?
- Write a function `f2` that does the same thing as `f` and that does not have this problem.

Question 2.3

Both functions `f` and `g` contain matches on option types in their last clause. These can be removed and the program streamlined by using `Option.map` and/or `Option.defaultValue`.

Create two functions `fOpt` and `gOpt` that behave the same as `f` and `g` respectively but with their match-statements replaced by shorter code that uses the library functions mentioned above. If you are unclear on what the library functions do, then look them up.

Question 2.4

Only one of the functions `f` and `g` is tail-recursive. Which one and why? To make a compelling argument you should evaluate a function call of the tail-recursive function, similarly to what is done in Chapter 1.4 of HR, and reason about that evaluation. You need to make clear what aspects of the evaluation tell you that the function is tail recursive.

Create a tail-recursive version of `f` or `g`, whichever was **not** already tail recursive, using continuations. The function should be called `fTail` or `gTail` depending on which one you implement.

$$\frac{4}{2x \cdot (2x + 1) \cdot (2x + 2)}$$

Question 3.2

In this assignment we will generate infinite sequences of increasingly accurate versions of π . It's important to note that this is **not** sequences of decimals of π , but where each index `i` of the sequence contains the version of π that has had the Nilakantha series run `i` times. For example, the first five elements of your sequences will look like this (up to six decimal places):

```
seq [3.0M; 3.166667M; 3.133333M; 3.145238M; 3.139682M; 3.142713M; ...]
```

Create an infinite sequence `piSeq : seq<decimal>` containing the infinite sequence of approximations of π described above.

Question 3.3

The area of a circle is determined by the formula πr^2 , where `r` is the radius of the circle, and the volume of a sphere is determined by $\frac{4}{3}\pi r^3$.

Create two functions `circleArea float -> seq<decimal>` and `sphereVolume float -> seq<decimal>` that given a radius `r` returns a sequence, of circle areas and sphere volumes respectively, with the radius `r` using the approximations of `pi` from `piSeq`.

Example (rounded to fewer decimals for readability)

```
circleArea 2.5 = seq [18.750M; 19.79166667M; 19.58333333M; 19.6577381M; ...]
sphereVolume 2.5 = seq [62.50M; 65.97222222M; 65.27777778M; 65.52579365M; ...]
```

Question 3.4

Create a function `circleSphere : float -> seq<decimal * decimal>` that given a radius `r` returns a sequence of pairs of circle areas and sphere volumes of radius `r` using the approximations of `pi` from `piSeq`.

For this exercise you must use sequence comprehension and **not** `Seq.zip` or any similar function.

Example

```
circleSphere 2.5 = seq
  [(18.750M, 62.50M); (19.79166667M, 65.97222222M); (19.58333333M, 65.27777778M)
  ;
  (19.6577381M, 65.52579365M); ...]
```

Hint: use `circleArea` and `sphereVolume` from the previous exercise.

Question 3.5

Create a parallelised version of Question 3.1 `parallelPi : uint64 -> uint64 -> decimal` that given two unsigned integers `numberOfProcesses` and `iterationsPerProcess` returns the value of π as approximated by running the Nilakantha series

`numberOfProcesses * iterationsPerProcess + 1` times (similar to Question 3.1) using `numberOfProcesses` processes and having each process apply the Nilakantha series `iterationsPerProcess` times, summing their results, and finally adding `3.0`.

This means that `calculatePi x = parallelPi 1UL x` and that `parallelPi x y = parallelPi y x`.

Hint: Consider `parallelPi 2UL 5UL`. A good solution, which also scales, is to have the first process calculate elements `1..5` from the Nilakantha series, have the second process calculate elements `6..10`, and finally adding the results to `3.0M` (element `0`).

Hint: Use `Async.Parallel` to collect the results from the individual processes.

Use `#time` to compare the running times of `calculatePi` and `parallelPi`. When does parallelism pay off? Motivate your answer with examples.

4: Programmable typewriters (25%)

Turing machines are used as models of computation. There is a close to infinite well of theory to dig into here, some of which was covered in your Discrete Mathematics course, but for the purposes of this assignment abstract away from that and just think of them as typewriters (which we call them from now on).

A typewriter has an infinite tape in both directions of the head of the typewriter and at any point in time it can either write a symbol to the tape at the position of the head, erase a symbol from the tape (which is the same as writing an empty symbol to the tape) **and** move the head one direction either to the left or to the right. Standing still is not permissible. A program execution is then a sequence of these actions. Symbols are parameters of the typewriter (a polymorphic type `'a`).

Important: Even though a tape is infinite in both directions it can only ever have a finite number of symbols written to it. All other spaces are assumed to be blank.

The purpose of this assignment is two-fold

1. Find a way to represent the typewriter in F# (Questions 4.1-4.3).
2. Evaluate programs, that we provide, consisting of the operations described above to modify the tape (Questions 4.4-4.6).

As an example of a typewriter and the symbols stored on the tape consider unary numbers where a sequence of `true` literals represent a number, and the `false` literal ends a number. A sequence of five occurrences of `true` followed by a single occurrence of `false` then represents the number 5.

To handle that tape elements can be blank we can use option types having `None` represent the blank parts of the tape. The following tape then represents the numbers `3` and `5` stored after each other (where `#` is blank, `T` is `Some true`, and `F` is `Some false`) and the head of the typewriter is placed at the start of the first number.

```
-----  
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
-----  
      ^ ^ ^
```

A program that adds two numbers would then transform the tape above to the following one

```
-----  
... # | # | T | T | T | T | T | T | T | T | F | # | # | ...  
-----  
      ^ ^ ^
```

where the numbers 3 and 5 have been added and 8 is stored on the tape.

Question 4.1

The head of the typewriter can either move to the left or to the right. This is modelled using an enumeration type.

```
type Dir = Left | Right
```

Create a type `Tape<'a>` for tapes. Recall that you will need a position for the head of the typewriter and an infinite tape to the left and the right. Also, for full marks, it must be possible to move the head of the typewriter in constant time as well as reading from and writing to the tape (only at the position of the head of the typewriter) in at most logarithmic time, but preferably constant time.

Hint: Even though tapes are infinite in both directions, infinite data structures like infinite sequences are a bad fit as there can at any point in time only be a finite number of symbols on the tape - the rest are blank.

Question 4.2

Write a function `tapeFromList : list<'a option> -> Tape<'a>` that given a list of symbol options `osyms` returns a tape with the elements from `osyms` stored in order with the head of the typewriter at the first element of `osyms`.

Example

```
tapeFromList [Some true; Some true; Some true; Some false;
              Some true; Some true; Some true; Some true; Some true; Some false] =

-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
      ^^^
```

```
tapeFromList [Some true; Some true; Some true; None;
              Some true; Some true; Some true; Some true; Some true] =

-----
... # | # | T | T | T | # | T | T | T | T | T | # | # | ...
-----
      ^^^
```

Write a function `tapeToList : Tape<'a> -> list<'a option>` that given a tape `tape`, regardless of where the head of the typewriter is, returns a list of the symbols on `tape` such that all leading and trailing blank spaces are cut.

Example

```
tapeFromList

-----
... # | # | T | T | T | # | F | # | T | T | T | T | T | F | # | # | ...
-----
      ^^^

=

[Some true; Some true; Some true; None; Some false; None;
 Some true; Some true; Some true; Some true; Some true; Some false] =
```

Question 4.3 (optional)

This exercise is optional but it is here to help. The purpose here is to construct functions that will be useful for Question 4.4 and you will get full marks by just solving that. However, the functions described here are useful and will provide partial credit on their own. Since they are optional they may be changed or even removed from the code skeleton file.

Moving the typewriter head

Write a function `moveHead : Dir -> Tape<'a> -> Tape<'a>` such that given a direction `dir` and a tape `tape` moves the head of `tape` in the direction `dir`.

Example

```
moveHead
Left
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
              ^^^
=
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
              ^^^
```

```
moveHead
Right
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
              ^^^
=
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
              ^^^
```

Reading from the tape

Write a function `readTape : Tape<'a> -> 'a option` that given a tape `tape` returns whatever symbol that the typewriter head points to, and `None` if it is blank.

Example

```
readTape
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
               ^^^
= Some false
```

```
readTape
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
               ^^^
= Some true
```

```
readTape
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
               ^^^
= None
```

Writing to the tape

Write a function `writeTape : 'a option -> Tape<'a> -> Tape<'a>` that given a symbol option `osym` and a tape `tape` writes the symbol `osym` to the point on the tape that the head of the typewriter is pointing to. If `osym` is `None` then that is equivalent to erasing a symbol from the tape.

Example

```
writeTape
  Some true
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
                        ^^^
=
-----
... # | # | T | T | T | T | T | T | T | T | T | F | # | # | ...
-----
                        ^^^
```

```
writeTape
  None
-----
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
-----
                        ^^^
=
-----
... # | # | T | T | T | # | T | T | T | T | T | F | # | # | ...
-----
                        ^^^
```


Question 4.4

Recall that at any point in time we can either write a symbol to the tape at the position of the head (or erase the symbol at the position of the head) **and** move the head one direction either to the left or to the right. To handle control flow we allow each action to give the name of the next action to be executed (like goto-statements or labelled switches or breaks). Informally this means that:

1. Programs are maps from names of actions (strings) to actions
2. Actions are maps from symbols on the tape (representing the symbol read) to
 - A label representing the name of the next action to be executed
 - The symbol to replace the read symbol (this can be the same symbol in which case the tape is left unchanged)
 - The direction in which to move the typewriter head after the write-operation has been done

Formally, this is how programs and actions are defined in F#.

```
type Action<'a when 'a : comparison> =  
    Act of Map<'a option, string * 'a option * Dir>  
type Program<'a when 'a : comparison> =  
    Prog of Map<string, Action<'a>>
```

Given a program `prog` a starting label `startLabel`, and a tape `tape` execution is handled as follows

- Lookup `startLabel` in `prog` to obtain an action `act`, if no such action exists then terminate normally returning `startLabel` and `tape`.
- Read from `tape` and look up the symbol in `act`, obtaining a new label `nextLabel`, a new symbol `newSymbol`, and a direction `dir`. If the symbol does not occur in `act` then terminate abnormally.
- Write `newSymbol` to `tape`
- Move the typewriter head on `tape` one step in direction `dir`
- recurse with the new starting state `nextLabel` and the new tape.

For our purposes we will handle normal and abnormal termination using option types.

Evaluating an action

Write a function

```
evalAction<'a> : when 'a : comparison> :  
  Action<'a> -> Tape<'a> -> (string * Tape<'a>) option
```

that given an action `act` and a tape `tape` performs `act` on `tape` and if successful returns the next label and the new tape, and `None` otherwise.

Example

For these examples we write `[k1 |-> v1; k2 |-> v2; ...]` to be a map where `k1` points to the value `v1`, `k2` points to the value `v2` and so on.

```
evalAction  
  Act ([Some false |-> ("nextLabel", Some false, Left)])  
-----  
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
-----  
          ^^^  
  
=  
Some (  
  "nextLabel",  
  -----  
  ... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
  -----  
          ^^^  
)
```

```
evalAction  
  Act ([Some false |-> ("anotherLabel", Some true, Right)])  
-----  
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
-----  
          ^^^  
  
=  
Some (  
  "anotherLabel",  
  -----  
  ... # | # | T | T | T | T | T | T | T | T | T | F | # | # | ...  
  -----  
          ^^^  
)
```

evalAction

```
Act ([Some false |-> ("yetAnotherLabel", None, Right)])
```

```
-----  
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
-----
```

^^^

=

```
Some (
```

```
  "yetAnotherLabel",
```

```
-----  
... # | # | T | T | T | # | T | T | T | T | T | # | # | ...  
-----
```

^^^

```
)
```

evalAction

```
Act ([Some true |-> ("label", None, Right)])
```

```
-----  
... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...  
-----
```

^^^

```
= None
```

Evaluating a program

Write a function

```
evalProgram<'a when 'a : comparison> :
  Program<'a> -> string -> Tape<'a> -> (string * Tape<'a>) option
```

That given a program `prog`, a start label `startLabel` and a tape `tape` executes `prog` on `tape` using `startLabel` as the first action.

Example

```
evalAction
  Prog
    ([ "runningLabel" |->
      Act ([Some false |-> ("runningLabel", Some false, Right);
          Some true |-> ("runningLabel", Some true, Right);
          None |-> ("endLabel", None, Left)
        ])
      ] )
    "runningLabel"
  -----
  ... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
  -----
               ^^^

=
Some
  (
    "endLabel",
    -----
    ... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
    -----
                           ^^^
  )
```

evalAction

Prog

```
(["runningLabel" |->
  Act ([Some false |-> ("runningLabel", Some true, Right);
        Some true |-> ("runningLabel", Some false, Right);
        None |-> ("endLabel", None, Left)
      ])
  )
```

)

"runningLabel"

... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...

^^^

=

Some

```
(
  "endLabel",
```

... # | # | T | T | T | T | F | F | F | F | F | T | # | # | ...

^^^

)

evalAction

Prog

```
(["runningLabel" |->
  Act ([Some false |-> ("runningLabel", Some true, Right);
        Some true |-> ("runningLabel", Some false, Right);
        None |-> ("endLabel", None, Right)
      ])
  )
```

)

"runningLabel"

... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...

^^^

=

Some

```
(
  "endLabel",
```

... # | # | T | T | T | T | F | F | F | F | F | T | # | # | ...

^^^

)

```
evalAction
  Prog
    ([ "runningLabel" |->
      Act ([Some false |-> ("runningLabel", Some false, Right);
          Some true |-> ("runningLabel", Some true, Right);
          None |-> ("endLabel", None, Left)
        ])
      ])
    "NonExistingLabel"
  -----
  ... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
  -----
                                     ^^^

=
Some
  (
    "NonExistingLabel",
    -----
    ... # | # | T | T | T | F | T | T | T | T | T | F | # | # | ...
    -----
                                     ^^^

  )
```

Question 4.5

Since `evalProgram` returns the name of the ending state as well as a resulting tape it is possible to construct a joint evaluation function for two programs (similar to how `>>` works) by executing one program from a starting tape and a starting state, and executing another program from the end state and resulting tape of the evaluation of the first program.

Create an infix function `|>>|` with type

```
(string -> Tape<'a> -> (string * Tape<'a>) option) ->
(string -> Tape<'a> -> (string * Tape<'a>) option) ->
(string -> Tape<'a> -> (string * Tape<'a>) option)
  when 'a : comparison
```

where the intent is that given two programs `prog1` and `prog2`

`evalProgram prog1 |>>| evalProgram prog2` returns an evaluation function of composing the executions of programs `prog1` and `prog2`.

This assignment is not available in your skeleton as that would give away how to define infix functions ;).

To get full marks on this assignment you must use railway-oriented programming or computation expressions (no matches on options within `|>>|`). The option monad, or the computation expression,

defined below come in handy. You may use either one and you do **not** have to use both.

Option monad

```
let ret = Some
let bind f =
  function
  | None    -> None
  | Some x -> f x

let (>=>) x f = bind f x
```

Computation expressions

```
type opt<'a> = 'a option

type OptBuilderClass() =
  member t.Return (x : 'a) : opt<'a> = ret x
  member t.ReturnFrom (x : opt<'a>) = x
  member t.Bind (o : opt<'a>, f : 'a -> opt<'b>) : opt<'b> = bind f o

let opt = new OptBuilderClass()
```

Question 4.6

Create a function

```
combineProgramEvaluations :
  (string -> Tape<'a> -> (string * Tape<'a>) option) list ->
  string -> Tape<'a> -> (string * Tape<'a>) option
```

that given a list of evaluation functions `evaluationFunctions` combines all of them in order using `|>>|`.

Testing your typewriter

To help you test your typewriter we have provided a small calculator for unary numbers.

The function `addTwoUnaryNumbers : int -> int -> unit` tests your `evalProgram` function. Successful run will, for instance, look as follows.

```
addTwoUnaryNumbers 3 4;;  
Adding numbers [3; 4] successful : true
```

whereas an unsuccessful attempt would output

```
addTwoUnaryNumbers 3 4;;  
Adding numbers [3; 4] successful : false
```

The function `addThreeUnaryNumbers : int -> int -> int -> unit` tests your `|>>|` function, and the `addSeveralUnaryNumbers : int list -> unit` tests your `combineProgramEvaluations` function. Do note that `addThreeUnaryNumbers` is commented out since `|>>|` does not appear in the skeleton. Just remove the comments to run the code.

You are more than welcome to change these functions if you require additional debug output, but you are not expected to spend time understanding these programs. They are only there to help.